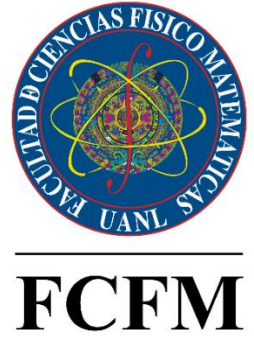




FACULTAD DE CIENCIAS FISICO MATEMATICAS



Laboratorio de Programación Orientada a Objetos

Profesor Jorge Alberto Islas Pineda

Laboratorio 3

Encapsulamiento y modificadores de acceso

Angel Gael Jimenez Mendieta
2096042

Introduccion

La programación orientada a objetos trata de imitar cómo funcionan las cosas en el mundo real. Cuando diseñamos un sistema, no solo se trata de que el código compile y ya, sino de que sea seguro y fácil de mantener a largo plazo. Aquí es donde entran en juego dos conceptos que van de la mano y que son la base para que nuestros proyectos no terminen siendo un dolor de cabeza cuando crecen: el encapsulamiento y los modificadores de acceso.

EXPLICACION

Para entender el encapsulamiento, podemos pensar en cómo usamos cualquier aparato en el día a día. Si manejamos un coche, nosotros usamos el volante y los pedales, pero no necesitamos abrir el motor y mover los engranes con las manos para avanzar. De hecho, si intentáramos hacerlo, lo más probable es que arruinaríamos el vehículo. En lenguajes como Java, el encapsulamiento hace exactamente lo mismo: esconde el funcionamiento interno de nuestros objetos y protege sus datos para que nadie pueda modificarlos a la fuerza o introducir valores absurdos.

Pero, ¿cómo le decimos al código qué cosas deben estar protegidas y qué cosas pueden verse desde afuera? Ahí es donde entran los modificadores de acceso. Son como las configuraciones de privacidad de nuestra clase y nos permiten establecer las reglas del juego.

El más restrictivo de todos es `private`. Cuando le ponemos esta etiqueta a un atributo, estamos diciendo "esto es exclusivo de esta clase y nadie más puede tocarlo". Es la forma ideal de proteger los datos importantes. Para que otras partes del programa puedan interactuar con esa información de forma segura, usamos métodos públicos conocidos como `getters` y `setters`. Estos actúan como filtros o guardias de seguridad; por ejemplo, si alguien intenta decir que un automóvil tiene -5 puertas, el `setter` lo bloquea porque ya le programamos una validación lógica que impide que ese dato irreal se guarde.

Por otro lado tenemos `public`, que es todo lo contrario. Lo que sea público está abierto a que cualquier otra parte del programa lo llame o lo modifique. Normalmente dejamos este modificador para los métodos, ya que son la forma en la que otros objetos se comunican con el nuestro.

Finalmente está `protected`, que funciona como un punto intermedio muy útil cuando trabajamos con herencia. Es información privada para el mundo exterior, pero está disponible de forma directa para las "clases hijas". Es como un secreto familiar: no se lo cuentas a extraños, pero sí lo compartes con los miembros de tu propia familia.

Al combinar el encapsulamiento con un buen uso de los modificadores de acceso, logramos que nuestro código sea mucho más robusto. Evitamos que otras partes

del sistema modifiquen cosas por accidente y nos aseguramos de que los objetos siempre tengan datos coherentes. Aplicar estas reglas desde el principio es lo que diferencia a un código frágil de un diseño bien estructurado.

PREGUNTAS DE REFLEXION

1. ¿Cuál es la diferencia entre `private` y `protected`? ¿En qué situación usarías cada uno?

La diferencia principal es quien puede ver la informacion. Usas `private` cuando quieres que un dato sea exclusivo de esa misma clase y nadie mas lo toque. En cambio, usas `protected` cuando tienes herencia y necesitas que las clases "hijas" (como `Automovil` o `Avion`) puedan usar ese dato directamente, pero manteniendolo bloqueado para el resto del programa.

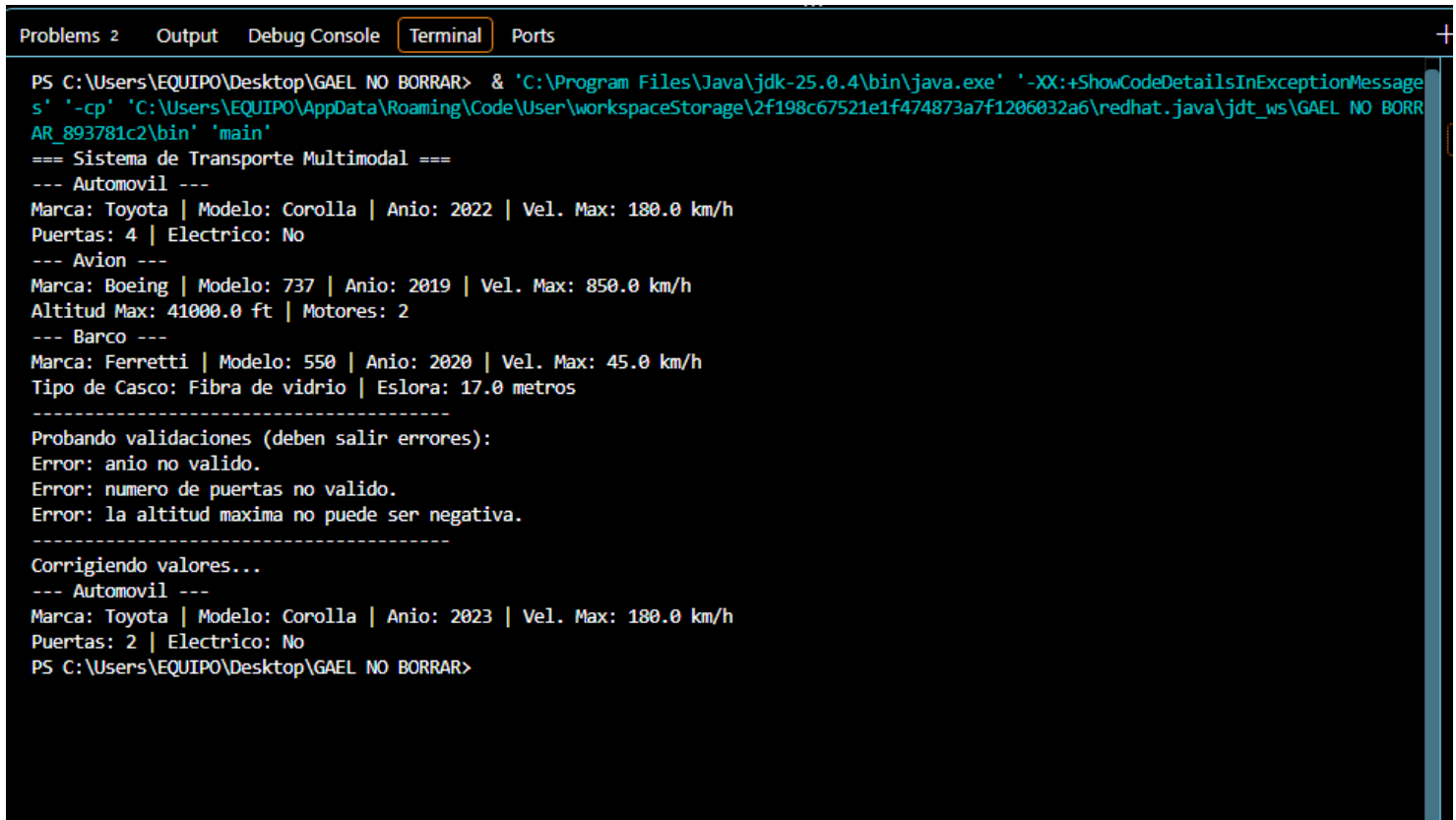
2. ¿Por qué `marca` y `modelo` no tienen `setter`? ¿Qué implica eso en el diseño?

No tienen `setter` porque son datos que no deberian cambiar una vez que el objeto se crea. En el diseño, esto implica que son atributos inmutables. Es decir, un coche nace siendo de una marca y modelo especifico desde el constructor y no tiene sentido que se transforme en otra cosa a mitad del camino, asi que los proteges evitando que se puedan modificar despues.

3. ¿Qué pasaría si un `setter` no tuviera validación y alguien pusiera un año negativo?

Si no hay validacion, el programa aceptaria el dato ciegamente. Alguien podria poner que un coche es del anio -500 y el sistema lo guardaria sin problemas. Esto arruinaria por completo la logica y la integridad de la informacion de tu aplicacion, volviendo el encapsulamiento inutil.

CODIGO FUENTE



```
PS C:\Users\EQUIPO\Desktop\GAEL NO BORRAR> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\EQUIPO\AppData\Roaming\Code\User\workspaceStorage\2f198c67521e1f474873a7f1206032a6\redhat.java\jdt_ws\GAEL NO BORRAR_893781c2\bin' 'main'
=== Sistema de Transporte Multimodal ===
--- Automovil ---
Marca: Toyota | Modelo: Corolla | Anio: 2022 | Vel. Max: 180.0 km/h
Puertas: 4 | Electrico: No
--- Avion ---
Marca: Boeing | Modelo: 737 | Anio: 2019 | Vel. Max: 850.0 km/h
Altitud Max: 41000.0 ft | Motores: 2
--- Barco ---
Marca: Ferretti | Modelo: 550 | Anio: 2020 | Vel. Max: 45.0 km/h
Tipo de Casco: Fibra de vidrio | Eslora: 17.0 metros
-----
Probando validaciones (deben salir errores):
Error: anio no valido.
Error: numero de puertas no valido.
Error: la altitud maxima no puede ser negativa.
-----
Corrigiendo valores...
--- Automovil ---
Marca: Toyota | Modelo: Corolla | Anio: 2023 | Vel. Max: 180.0 km/h
Puertas: 2 | Electrico: No
PS C:\Users\EQUIPO\Desktop\GAEL NO BORRAR>
```

CONCLUSION

En conclusion, aplicar el encapsulamiento y elegir bien los modificadores de acceso es como ponerle un filtro de seguridad a nuestro codigo. Nos ayuda a evitar errores accidentales, mantiene la informacion clave protegida y asegura que cada objeto se comporte exactamente como lo planeamos. Al final, adoptar estas buenas practicas es lo que nos permite construir programas mucho mas ordenados, seguros y faciles de actualizar en el futuro, sin miedo a que un pequeño cambio rompa todo el sistema.